VNAV 视觉导航课程实验

Lab3 无人机轨迹生成与跟踪控制

黄博宇 24020036026 | 中外计科一班 | 第8组

实验日期：2026年10月21日；报告日期：2026年10月23日

摘要

本实验补全了四旋翼控制器中的11处空白，在Ubuntu 20.04与ROS Noetic环境下完成定点悬停和圆轨迹跟踪。目标圆半径5米、高度2米。初期参数造成翻转，降低姿态反馈并调整位置、速度反馈后，飞行不再明显晃动。对44.64秒录制数据进行时间对齐，获得2233个位置样本，三维位置均方根误差（RMSE）为0.1052米，最大误差为0.1166米，偏航RMSE为9.524°。报告保留全部8张原始截图，并结合代码、曲线和理论分析说明哪些任务完成了、哪些偏差仍存在。阻力补偿完成了推导，但没有加入飞行代码。

1 引言：这个实验要解决什么

无人机想向侧面加速，通常要先倾斜机身，让旋翼推力带上水平分量。因此，“飞到哪里”和“机身怎样摆”要一起控制。控制器像一个不断纠偏的驾驶员：读取目标和实际状态，算出需要的力和力矩，再分给四个旋翼。力矩就是让机身转起来的作用。

表1 实验任务

任务	本次内容
D1	5个姿态转换接口小问
D2	旋翼配置分析、阻力补偿推导
D3	补全控制器、调参、悬停及绕圆验证

实验依据2023年Lab3题面和VNAV2021工程[1,2]。轨迹公式由工程提供，不需要另写路径规划算法。本报告按模板组织为引言、实验设置与步骤、结果、讨论、结论和参考文献。

2 实验设置与步骤

2.1 环境与目录

先确认系统和源码位置，避免后面在错误的工作空间编译。ROS可以理解为程序之间的通信系统，catkin负责把源码编译成可运行程序。图1保留了实际环境检查结果。

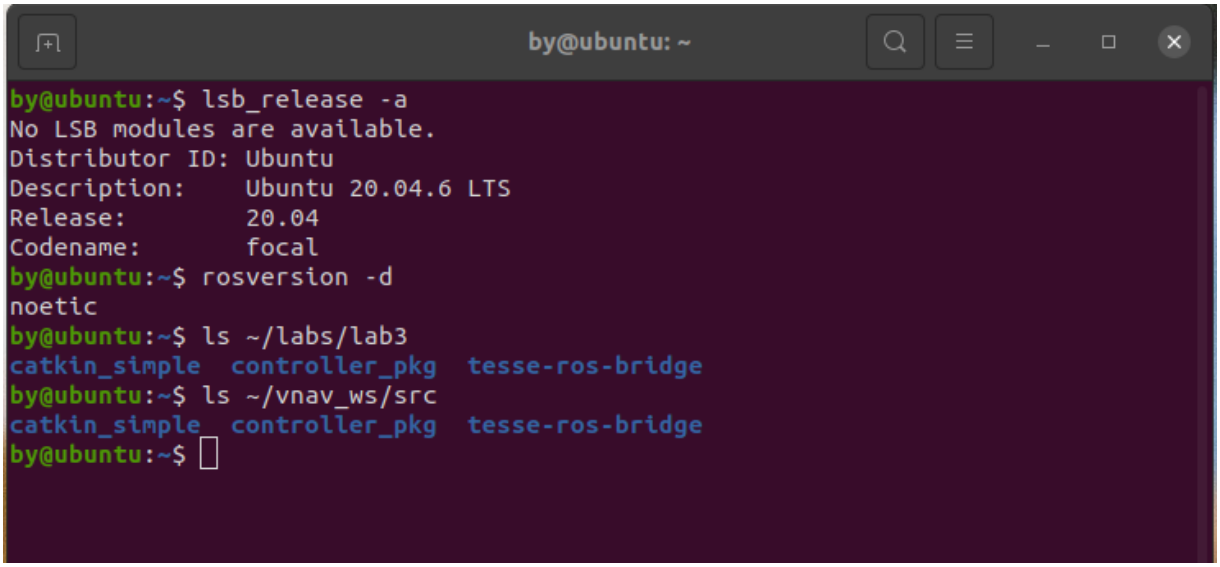


图1 Ubuntu、ROS版本及Lab3源码目录（原截图01）

表2 运行环境

项目	实际配置	用途
系统	Ubuntu 20.04.6 LTS，VMware	运行ROS与仿真器
ROS	Noetic	传递状态和控制指令
源码	VNAV2021	提供待补全控制器及轨迹程序
工作空间	~/vnav_ws	保存源码、编译结果与环境配置

~/labs保存下载的教学工程，~/vnav_ws/src才是实际编译使用的源码目录。~/vnav-builds/lab3保存仿真器。三者用途不同，改错目录会出现“代码改了，运行却没变”的情况。

2.2 原始工程首次编译

第一次编译用于检查依赖和工程结构，不等于控制器已经完成。即使模板中的控制逻辑还没补齐，工程也可能生成可执行文件。图2对应这个准备阶段。

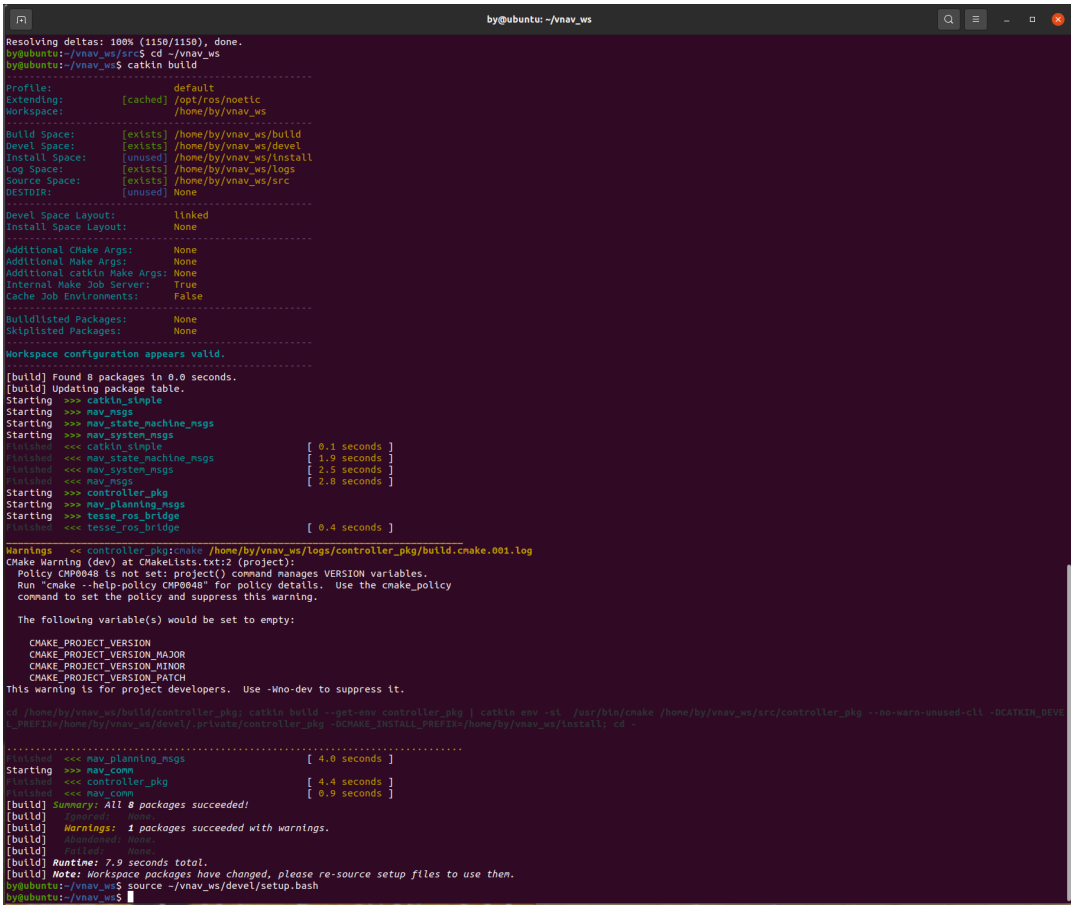


图2 原始工程首次catkin build成功记录（原截图02）

表3 主要依赖

依赖 / 工具	负责什么
catkin_simple	帮助组织教学包的编译
mav_comm	提供旋翼与规划相关消息
TESSE Python接口	支持仿真器与ROS桥接

检查重点是最终Summary中所有包成功、Failed为None。CMake策略警告未阻止这次构建；飞行是否正确，还需要后面的状态检查和实际测试。

2.3 仿真器与ROS桥接

TESSE负责模拟无人机受力后的运动；桥接程序负责“传话”。控制器把旋翼命令发给桥接，桥接交给仿真器，再把新的位置、速度和姿态传回来。图3是刚打开的场景，不是已经完成飞行的证明。



图3 TESSE Unity启动后的无人机初始场景（原截图03）

表4 程序分工

程序	输入	输出
traj_publisher	当前时间与模式开关	目标位置、速度、加速度和yaw
controller_node	目标状态与实际状态	四个旋翼转速
ROS桥接＋TESSE	旋翼指令	实际飞行状态
RViz	ROS状态消息	可观察的位姿和坐标显示

旋翼由控制器持续发命令驱动。R键用于把仿真器中的无人机重置到初始位置；调试时先停止控制器，再重置，避免旧指令继续作用。

2.4 状态显示与坐标系

RViz相当于观察窗口。图4中的箭头和坐标轴用于查看状态，不是实际机身外形。先确认状态在更新、坐标含义正确，再开始调参。

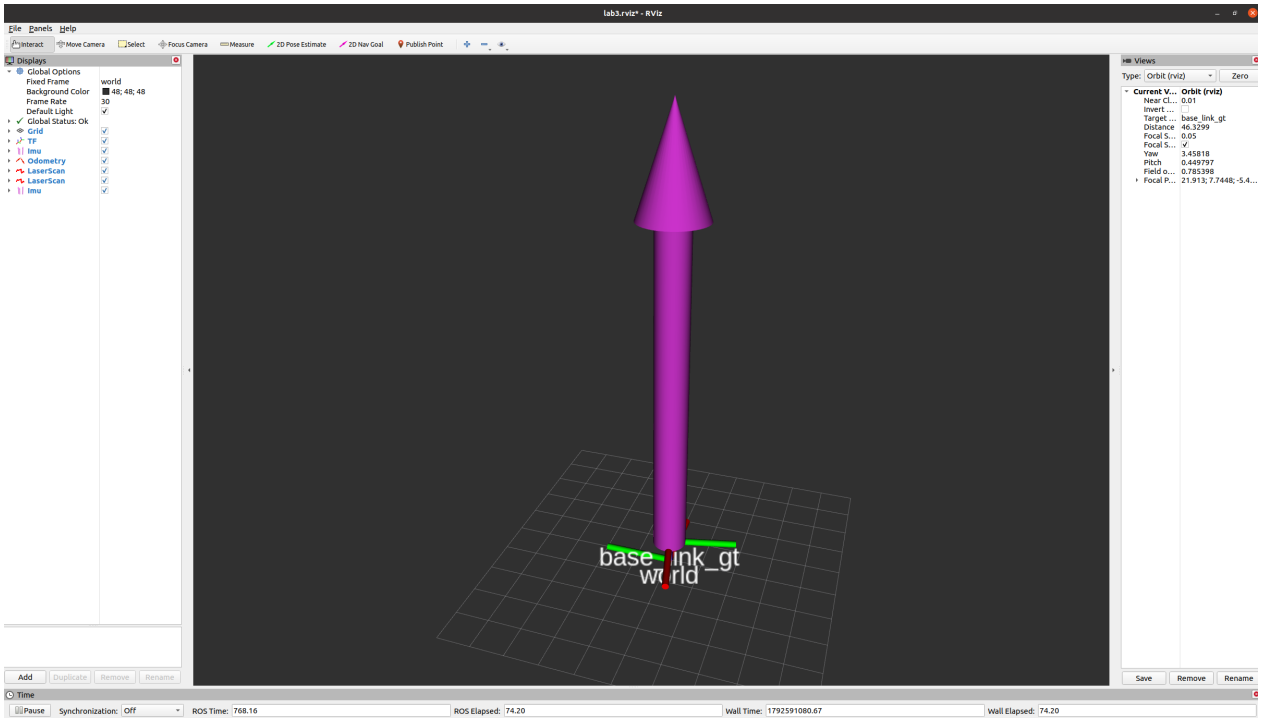


图4 RViz中的world、base_link_gt及状态显示（原截图04）

表5 坐标约定

名称 / 约定	通俗解释	本实验怎样使用
world	固定在场景里的坐标尺	位置和线速度用世界系
base_link_gt	跟着无人机一起转的坐标尺	表示机体姿态
z轴向上	高度增加为正	重力补偿取+mg e3
角速度转换	换一套方向描述同一个转动	omega=R^T omega_world

R将机体系向量转到世界系，R^T做反向转换。本实验的桥接把角速度放在世界系中，而力矩公式使用机体系角速度。这里的约定来自教学代码，不能对所有ROS里程计消息一概照搬。

2.5 补全任务与姿态接口

需要补全的是controller_node.cpp中的11处空白。traj_publisher.cpp已给出轨迹公式，只用于切换悬停和绕圆。表6把11处内容与用途对应起来。

表6 原模板的11处补全任务

编号	位置	负责什么
01	PART 1	声明订阅器、发布器、定时器和状态标志
02	PART 2	把通信对象连接到回调函数
03	PART 3.1	读取目标位置、速度、加速度
04	PART 3.2	读取目标yaw
05	PART 4	读取实际状态，转换角速度坐标系
06	PART 5.1	计算位置、速度误差
07	PART 5.2	根据所需合力构造期望姿态
08	PART 5.3	计算姿态和角速度误差
09	PART 5.4	计算总推力与三轴力矩
10	PART 5.5	计算旋翼有符号转速平方
11	PART 5.6	开平方并发布四个旋翼角速度

D1：5个接口答案

表7 D1接口问答

题目	核心答案
A1 消息转tf2	tf2::fromMsg(quat_msg, quat_tf);
A2 tf2转消息	geometry_msgs::Quaternion quat_msg = tf2::toMsg(quat_tf);
A3 标量分量	quat_tf.w();
B1 水平朝向	tf2::getYaw(quat_t);
B2 旋转矩阵	fromMsg读入Eigen四元数，再调用toRotationMatrix()

```
Eigen::Quaterniond quat_e;
tf2::fromMsg(quat_msg, quat_e);
Eigen::Matrix3d R = quat_e.toRotationMatrix();
```

四元数用4个数表示朝向。B2假设输入为有效单位四元数；非单位输入需先归一化。Eigen构造函数顺序为(w,x,y,z)，不能与ROS字段顺序混淆。完整头文件和答案见deliverable_1.txt。

2.6 核心代码：通信与目标读取

下面是补全01至04的核心语句。订阅器收到消息后调用相应函数；发布器发出转速；定时器持续触发控制计算。代码节选省略了类外框和重复模板注释，不能当作完整文件单独编译。

```
// 补全01：声明通信对象，以及控制启动前的状态标志。
ros::Subscriber desired_sub, current_sub;
ros::Publisher rotor_pub;
ros::Timer control_timer;
bool have_desired = false, have_current = false;

// 补全02：连接状态回调；每个队列保留最近排队的消息。
desired_sub = nh.subscribe("desired_state", 1,
                           &controllerNode::onDesiredState, this);
current_sub = nh.subscribe("current_state", 1,
                           &controllerNode::onCurrentState, this);
rotor_pub = nh.advertise<mav_msgs::Actuators>("rotor_speed_cmds", 1);
control_timer = nh.createTimer(ros::Duration(1.0 / hz),
                               &controllerNode::controlLoop, this);

// 补全03：在目标回调中先检查数组，再读取位置、速度和加速度。
if (des_state.transforms.empty() || des_state.velocities.empty() ||
    des_state.accelerations.empty()) return;
const auto& position = des_state.transforms[0].translation;
const auto& velocity = des_state.velocities[0].linear;
const auto& acceleration = des_state.accelerations[0].linear;
xd << position.x, position.y, position.z;
vd << velocity.x, velocity.y, velocity.z;
ad << acceleration.x, acceleration.y, acceleration.z;

// 补全04：拒绝无效四元数，归一化后读取目标偏航角。
const auto& q = des_state.transforms[0].rotation;
const double norm2 = q.x*q.x + q.y*q.y + q.z*q.z + q.w*q.w;
if (!std::isfinite(norm2) || norm2 < 1e-12) return;
tf2::Quaternion quat(q.x, q.y, q.z, q.w);
quat.normalize();
yawd = tf2::getYaw(quat);
have_desired = xd.allFinite() && vd.allFinite() && ad.allFinite()
               && std::isfinite(yawd);
```

表8 目标输入处理

处理	为什么这样做
先检查数组	空数组不能直接访问[0]
检查四元数并归一化	避免零长度或无效姿态参与计算
have_desired	收到有限值的目标后才允许启动控制

2.7 核心代码：读状态、求合力

补全05至07把消息中的状态变成控制量。误差按“实际减目标”定义，所以反馈项带负号。位置反馈负责拉回来，速度反馈像刹车，目标加速度则提前提供轨迹需要的力。

$$F = -k_x e_x - k_v e_v + mge_3 + ma_d$$

式（1）

```
// 补全05：在实际状态回调中读取位置、速度、姿态和角速度。
const auto& position = cur_state.pose.pose.position;
const auto& velocity = cur_state.twist.twist.linear;
const auto& q = cur_state.pose.pose.orientation;
Eigen::Quaterniond quat(q.w, q.x, q.y, q.z); // 注意：Eigen构造顺序为w、x、y、z。
if (!std::isfinite(quat.squaredNorm()) || quat.norm() < 1e-6) return;
quat.normalize();
R = quat.toRotationMatrix();
x << position.x, position.y, position.z;
v << velocity.x, velocity.y, velocity.z;
const auto& rate = cur_state.twist.twist.angular;
// 此桥接给出世界系角速度；力矩公式需要机体系角速度。
omega = R.transpose() * Eigen::Vector3d(rate.x, rate.y, rate.z);
have_current = x.allFinite() && v.allFinite() && R.allFinite()
               && omega.allFinite();

// 补全06至07：进入控制循环，先等到两类有效状态。
if (!have_current || !have_desired) {
  ROS_WARN_THROTTLE(2.0, "Waiting for valid current and desired states.");
  return;
}
ex = x - xd;  // 误差取实际减目标，因此反馈项前使用负号。
ev = v - vd;
// 世界系z轴向上，重力补偿为正的m*g*e3。
const Eigen::Vector3d force = -kx*ex - kv*ev + m*g*e3 + m*ad;
if (!force.allFinite() || force.norm() < 1e-6) return;
const Eigen::Vector3d b3d = force.normalized();
const Eigen::Vector3d heading(std::cos(yawd), std::sin(yawd), 0.0);
Eigen::Vector3d b2d = b3d.cross(heading);
if (b2d.norm() < 1e-6) return; // 方向退化时不做归一化，避免除以近零值。
b2d.normalize();
const Eigen::Vector3d b1d = b2d.cross(b3d);
Eigen::Matrix3d Rd;
Rd.col(0) = b1d;
Rd.col(1) = b2d;
Rd.col(2) = b3d;
```

合力方向决定期望机身z轴，再用目标yaw和叉积补齐另外两个方向，组成R_d。这里同时检查合力和叉积是否接近零，避免归一化时除以近零值。

这些检查用于避免部分无效输入和退化运算，不包含消息长期不更新时的完整超时保护。

2.8 核心代码：力矩与旋翼分配

补全08至11先算机身怎样转，再把总推力和三轴力矩分给旋翼。Vee把反对称矩阵转换为三维向量。有符号开平方保留转速方向，不能把负值一律截为零。

$$e_R = \frac{1}{2}(R_d^T R - R^T R_d)^{\vee}, \quad e_\Omega = \Omega$$

式（2）

$$M = -k_R e_R - k_\Omega e_\Omega + \Omega \times (J\Omega)$$

式（3）

```
// 配套实现：按仿真器编号建立分配矩阵，只在构造时求逆。
const double arm = d / std::sqrt(2.0); // X形机架在x、y方向的投影臂长。
Eigen::Matrix4d allocation;
allocation << cf, cf, cf, cf,
              arm*cf, arm*cf, -arm*cf, -arm*cf,
              -arm*cf, arm*cf, arm*cf, -arm*cf,
              cd, -cd, cd, -cd;
F2W = allocation.inverse();

// 补全08至09：计算姿态误差、推力与力矩。
er = 0.5 * Vee(Rd.transpose()*R - R.transpose()*Rd);
eomega = omega; // 题目允许的简化：省略目标角速度前馈。
const double thrust = force.dot(R.col(2));
const Eigen::Vector3d moment = -kr*er - komega*eomega
                               + omega.cross(J*omega);
Eigen::Vector4d wrench;
wrench << thrust, moment.x(), moment.y(), moment.z();

// 补全10：求每个旋翼的有符号转速平方，保留正负号。
const Eigen::Vector4d rotor_signed_square = F2W * wrench;
if (!rotor_signed_square.allFinite()) {
  ROS_ERROR_THROTTLE(1.0, "Non-finite rotor command: stop and inspect states.");
  return;
}

// 补全11：发布四个旋翼角速度，不是平方值。
mav_msgs::Actuators command;
command.header.stamp = ros::Time::now();
command.angular_velocities.resize(4);
for (int i = 0; i < 4; ++i) {
  command.angular_velocities[i] = signed_sqrt(rotor_signed_square(i));
}
rotor_pub.publish(command);
```

本仿真器1至4号位置为(+x,+y)、(-x,+y)、(-x,-y)、(+x,-y)，偏航符号为+、−、+、−。臂长要投影为d/√2。发布的单位是rad/s，而不是rpm或转速平方。

本次按题目允许的简化令e_omega=omega，省略目标角速度前馈。质量、惯量和推力系数沿用源码，未独立标定。

2.9 补全后重新编译

源码改完后再次执行catkin build。图5与图2属于两个阶段：图2检查原始工程，图5检查补全后的程序。编译成功说明代码能够生成程序，还不能替代飞行测试。

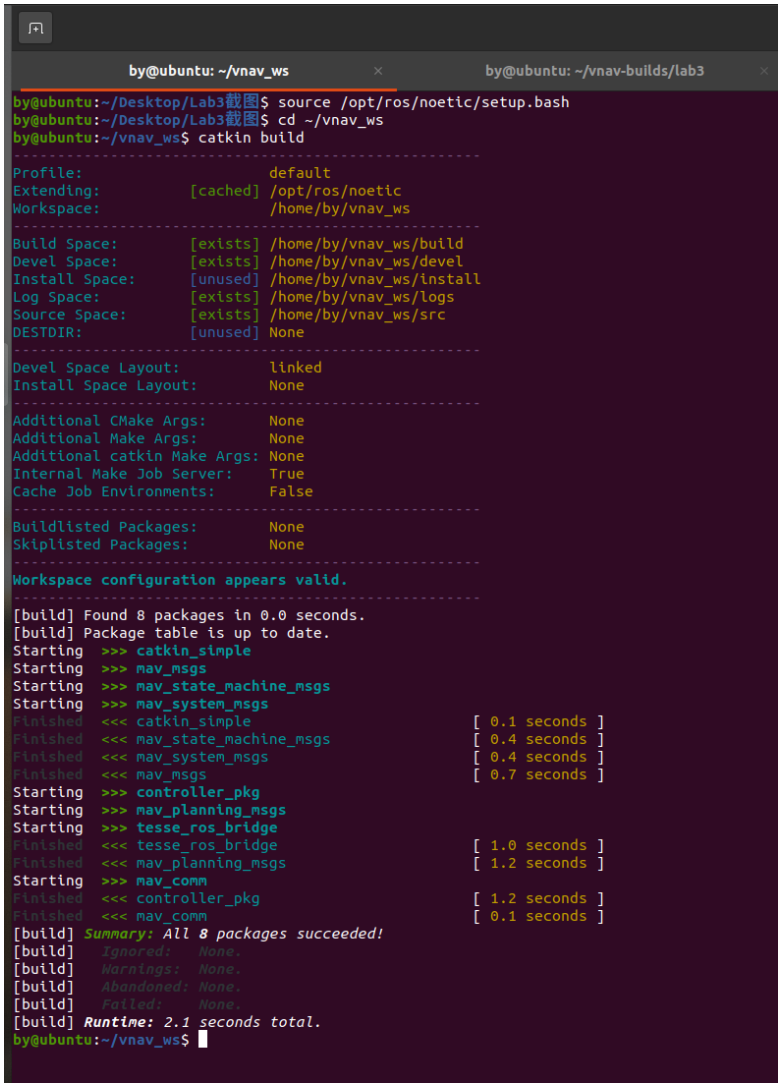


图5 补全控制器后整个工作空间编译成功（原截图05）

表9 补全后的构建检查

检查	本次结果
构建摘要	8个包成功，Failed为None
环境加载	编译后重新source工作空间setup.bash
最终参数	kx=2、kv=3、kr=3、komega=1

以后只改launch中的参数不需要重新编译；改C++源码或STATIC_POSE宏需要重新编译并重启节点。否则可能运行的仍是旧程序。

2.10 定点悬停：先让无人机站稳

先把STATIC_POSE设为1，目标为(0,0,2)米、yaw为45°。悬停让目标位置不再移动，便于分清是控制方向不对，还是参数纠偏太猛。图6显示悬停模式的位姿，图7用终端核对实际高度。

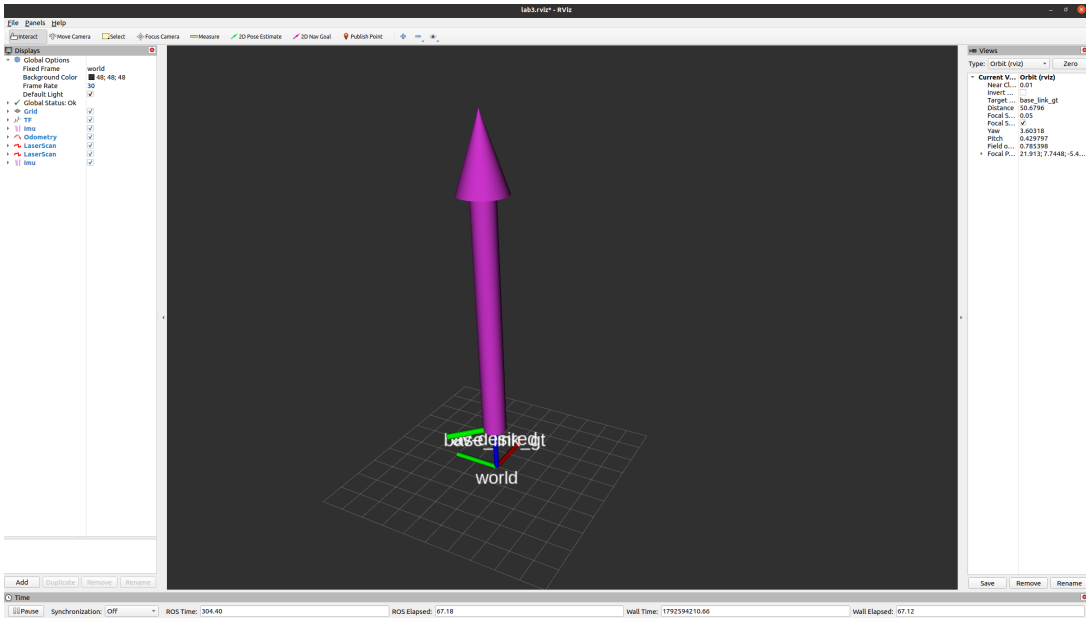


图6 STATIC_POSE定点模式下的目标与实际位姿（原截图06）

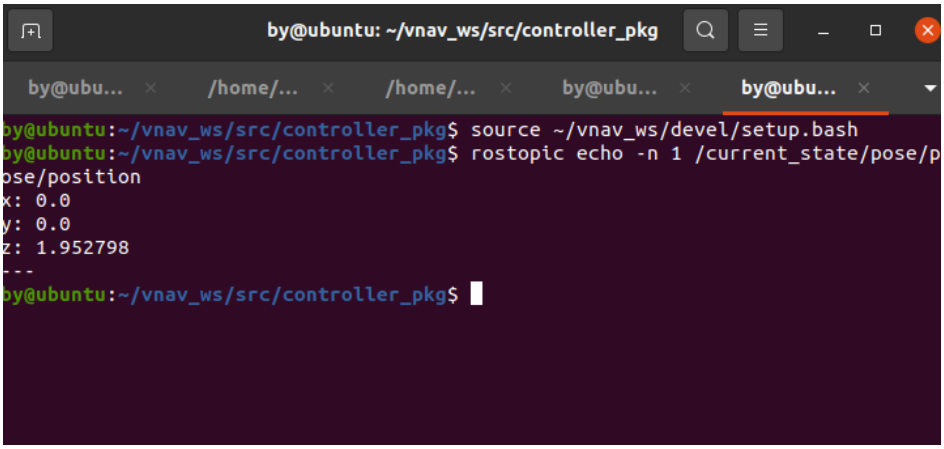


图7 悬停高度z=1.952798米（原截图06B）

表10 悬停读数的含义

读数	说明
目标2米	希望保持的高度
图7为1.952798米	比目标低约4.72厘米
另一次为1.957025米	来自另一个时刻，不与图7混为同一读数

原截图06B文件名含1.957，但图中的真实值是1.952798。报告以图中读数为准。单次读数只能说明当时的位置，不是整段悬停的RMSE。

2.11 圆轨迹：再让目标动起来

悬停稳定后，将STATIC_POSE改为0，重新编译并重置仿真器，再启动控制器。图8保留了绕圆时的RViz画面；完整运动过程由视频提供，单张截图不能证明一整圈都稳定。

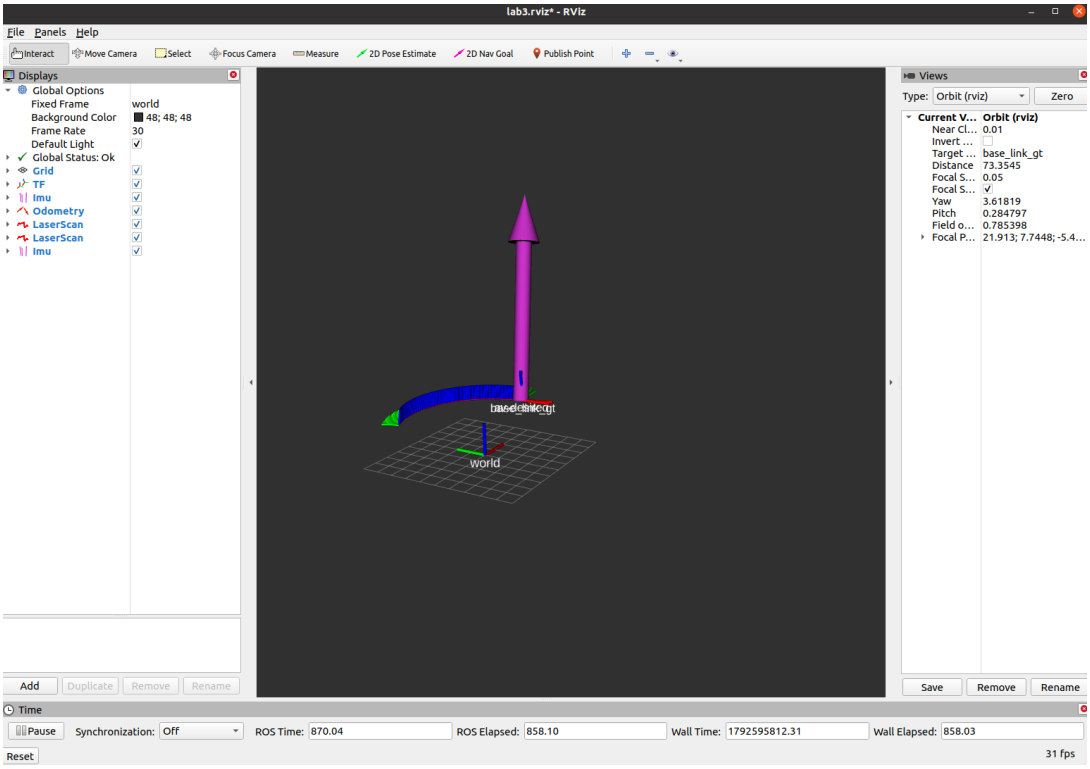


图8 RViz稳定绕圆状态与目标 / 实际位姿（原截图07）

表11 给定轨迹及其导数；单位依次为米、米/秒、米/秒^2

方向	位置	速度	加速度
x	5sin(t/2)	2.5cos(t/2)	-1.25sin(t/2)
y	5cos(t/2)	-2.5sin(t/2)	-1.25cos(t/2)
z	2	0	0

yaw_d=-t/2，目标半径5米、高度2米，水平速度2.5米/秒，每圈约12.57秒。轨迹发布器同时给位置、速度和加速度，控制器不只是追着一个位置点跑，还能提前准备转弯需要的力。

截图中的箭头表示位姿，不应把箭头尾迹直接当作数据计算的完整轨迹。下一节的轨迹图来自bag中的位置数据。

3 实验结果

3.1 期望轨迹与实际轨迹

录制段覆盖约3.55圈。图9给出实际与期望位置，表12汇总同一段数据的统计结果。三维误差同时考虑前后、左右和高度偏差，不只检查圆的大小。

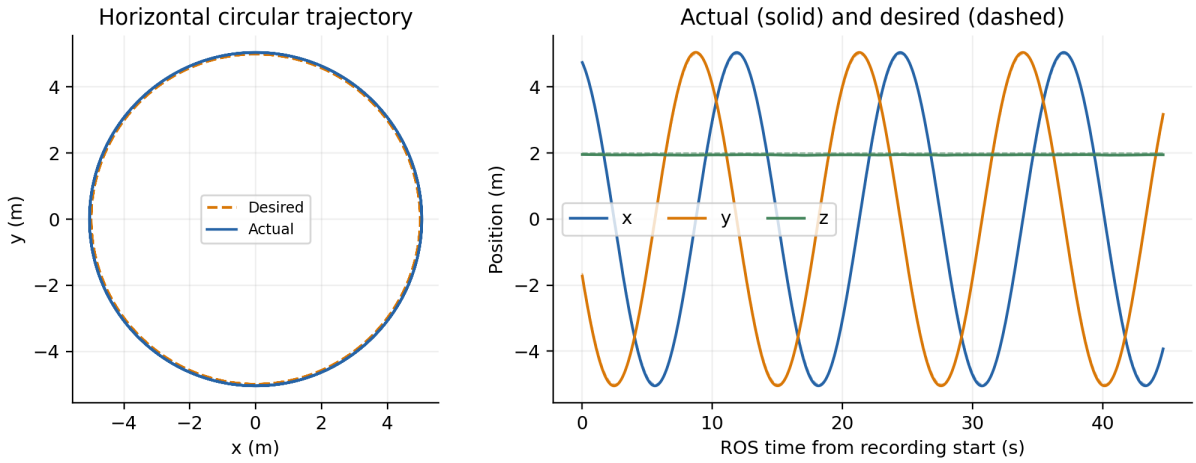


图9 bag数据计算的实际与期望轨迹

表12 圆轨迹跟踪统计

指标	实测结果
共同时间段	ROS时间933.72–978.36秒，共44.64秒
位置样本 / 实际圈数	2233点 / 约3.552圈
三维位置RMSE / 平均误差	0.1052米 / 0.1049米
最大 / 95分位三维误差	0.1166米 / 0.1134米
x / y / z分量RMSE	0.0631 / 0.0632 / 0.0556米
平均高度 / 高度范围	1.9446米 / 1.9318–1.9570米
平均半径 / 径向RMSE	5.0466米 / 0.0466米
偏航RMSE	9.524°
旋翼指令范围	29.17–63.60 rad/s

这些值描述本次稳定录制段，不包括对所有起飞过程或所有初始状态的验证。

3.2 误差怎样计算

目标和实际消息不是同时到达的，不能把“第100条”直接对上“第100条”。先对齐时间，再逐点求差，才有可解释的误差。插值就是利用相邻两个目标时刻，估计中间时刻的目标位置。

表13 时间对齐流程

步骤	具体处理
1	实际状态按header时间戳排序、去重
2	目标按bag记录的ROS到达时间插值
3	只取共同时间范围，得到2233个位置样本
4	计算每个时刻实际位置减目标位置
5	全部对齐样本参与统计，不按误差大小删点

$$\mathrm{RMSE}_{3D}=\sqrt{\frac{1}{N}\sum_{i=1}^{N}\left\|x_i-x_{d,i}\right\|^2}$$

式（4）

表14 指标含义

指标	怎样理解
三维RMSE	整段误差的汇总值，大偏差权重更高
平均误差	各时刻三维误差长度的平均值
最大误差	录制段里最大的单次偏差
95分位误差	约95%的样本不超过这个值
径向RMSE	只看圆半径，不包含全部相位和高度偏差

偏航角先展开再插值，最后把角度差折回[-π，π]，避免越过±180°时产生假跳变。bag中实际状态2235条、目标与旋翼指令各4404条、时钟2203条；消息数不相等本身不能证明丢包。

视频为40秒、15帧/秒，共600帧，使用现实时间；上面的44.64秒来自ROS仿真时间，两者分开统计。

3.3 误差随时间怎样变化

轨迹看起来接近，并不代表没有偏差。图10把各方向误差和偏航误差单独画出来，便于看清持续偏差和短时变化。原始局部尖峰被保留，没有为了画面好看而平滑掉。

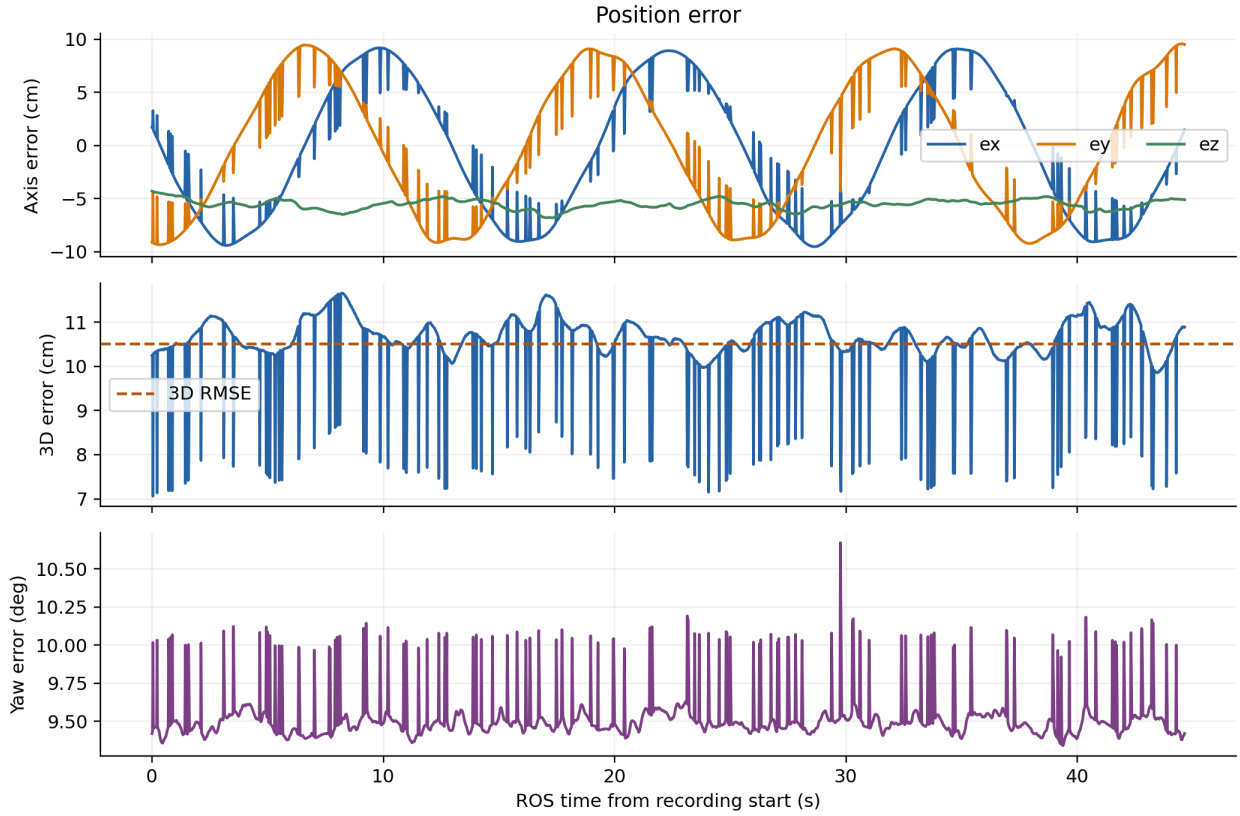


图10 三轴位置误差、三维误差及偏航误差

表15 曲线中的主要现象

观察	支持的结论
位置误差没有持续增大	这段录制内未出现持续发散
平均高度低5.54厘米	高度接近目标，但仍存在偏差
平均半径大4.66厘米	实际圆略大，不能只凭肉眼判断准确
偏航误差接近常值	朝向跟踪存在持续滞后

短时变化还会受到消息到达与时间戳关系的影响。本实验没有独立测量这些时延，因此不能把每个尖峰都解释成真实位置突然跳变。

3.4 调参过程与旋翼指令

调参从“翻转”走到“能悬停但晃”，最后才到“无明显晃动”。主要经验是纠偏不能一味加大：太猛的动作也会把机身带过头。表16记录实际经过，不把观察性的调试写成严格对照试验。

表16 三阶段调参记录

阶段	kx / kv / kr / komega	观察
初次实现	6 / 4 / 30 / 10	翻转，旋翼命令剧烈变化
降低姿态反馈	6 / 4 / 3 / 1	能够悬停，晃动仍逐渐明显
最终采用	2 / 3 / 3 / 1	悬停无明显晃动，完成绕圆录制

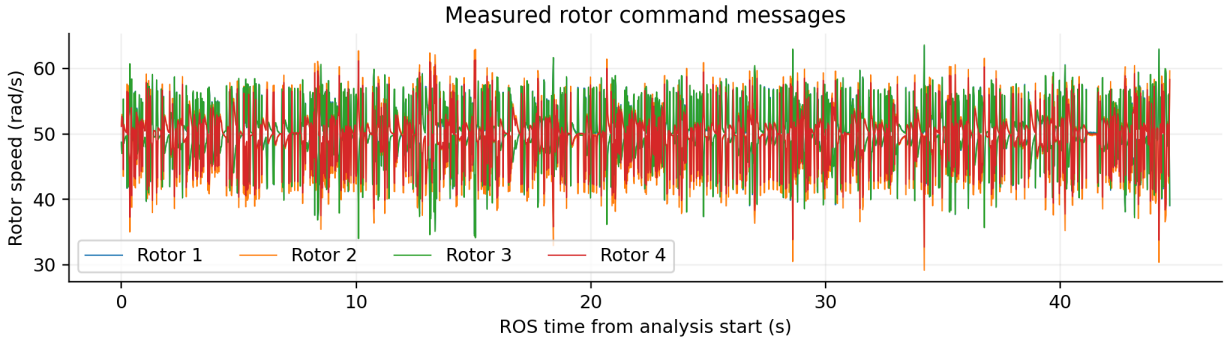


图11 四个旋翼的指令转速，不是独立测得的实际转速

表17 四个增益的分工

参数	最终值	作用
kx	2	按位置偏差把无人机拉回目标
kv	3	抑制速度偏差，像刹车
kr	3	纠正机身方向
komega	1	抑制过快转动

源码取m=1、c_f=0.001时，理想水平悬停的等速估算为√[mg/(4c_f)]=49.52 rad/s，落在记录指令29.17–63.60 rad/s范围内。这只检查量级，不能证明参数已标定。各组试验条件并未严格统一，也不能据此声称最终参数最优。

4 讨论

4.1 D2A：为什么旋翼配置会影响控制

D2A比较两种配置：正常四旋翼，以及更换3、4号桨并反转旋向后的四旋翼。关键不是“还能不能产生升力”，而是总推力和三轴力矩能否分别调节[1,3]。

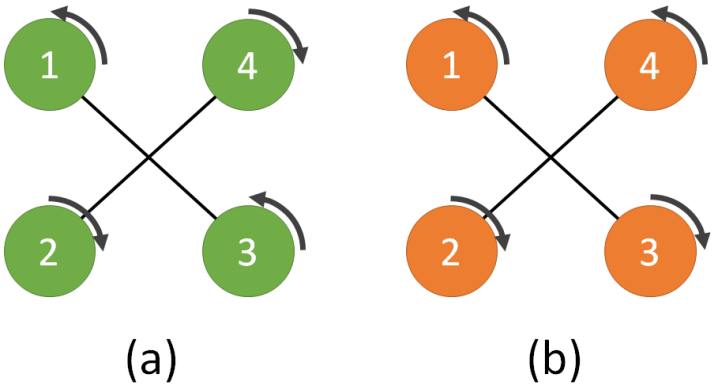


图12 理论题给出的正常配置a与改变后的配置b

仅本题采用图中编号：x向右、y向上，旋翼依次在(-a,a)、(-a,-a)、(a,-a)、(a,a)，a=d/√2。令u_i=ω_iˆ2，则[f,τ_x,τ_y,τ_z]ˆT=Fu。c_f表示升力系数，c_d表示偏航力矩系数。不要把这里的编号直接用到仿真代码。

$$F_a = \begin{bmatrix} c_f & c_f & c_f & c_f \\ ac_f & -ac_f & -ac_f & ac_f \\ ac_f & ac_f & -ac_f & -ac_f \\ -c_d & c_d & -c_d & c_d \end{bmatrix} \qquad F_b = \begin{bmatrix} c_f & c_f & c_f & c_f \\ ac_f & -ac_f & -ac_f & ac_f \\ ac_f & ac_f & -ac_f & -ac_f \\ -c_d & c_d & c_d & -c_d \end{bmatrix}$$

式（5） 两种配置的控制分配矩阵

表18 旋翼配置分析

配置	数学结果	控制含义
a	rank=4；|det|=16aˆ2c_fˆ3c_d	四个控制量可以独立指定
b	rank=3；τ_z=-c_d τ_x/(ac_f)	滚转与偏航被固定关系耦合

当a、c_f、c_d为正，配置a的行列式不为零。配置b前三行独立，但第四行是第二行的固定倍数，秩降为3：改滚转，偏航就被带着变。因此不能任意独立跟踪位置和yaw；某些满足约束的特殊运动仍可能实现。

4.2 D2B：怎样抵消空气阻力

阻力像一只拖住无人机的手。知道它往哪拖，就可在期望合力中补上反方向的力。题目模型按机体方向定义，因此先将速度转到机体系[1]：

$$v^b = R^T v, \quad D = \mathrm{diag}(0.1, 0.1, 0.2), \quad F^b = -D\,(v^b \odot |v^b|)$$

式（6）

⊙表示对应分量分别相乘。世界系z向上时，重力、旋翼推力和阻力满足：

$$m\dot{v} = -mge_3 + fRe_3 + RF^b$$

式（7）

误差取e_x=x-x_d、e_v=v-v_d。估计阻力转回世界系后，从基础合力中减去：

$$A = -k_x e_x - k_v e_v + mge_3 + ma_d - R\widehat{F}^b$$

式（8）

由补偿合力构造期望机身z轴，总推力取当前推力轴上的投影：

$$b_{3d} = A/\|A\|, \quad f = A^T R e_3$$

式（9）

若姿态跟踪足够准确，使推力向量接近A，代回动力学得到近似误差方程：

$$m\ddot{e}_x \simeq -k_x e_x - k_v e_v + R(F^b - \widehat{F}^b)$$

式（10）

表19 推导与实现的界限

内容	本次范围
补偿推导	完成；估计准确时可抵消主要阻力项
飞行实现	未加入补偿，未做补偿开关对比
验证方法	固定轨迹、参数和初始状态，比较误差与指令

模型误差和速度噪声会留下残差，因此不能只看公式就保证效果，也不能把本次的10.52厘米误差归因于阻力补偿。

4.3 结果为什么还有偏差

最明显的剩余问题是朝向滞后。本次省略了目标角速度前馈，持续转向时，角速度阻尼需要靠非零姿态误差产生的力矩平衡。近似纯偏航稳态关系为：

$$k_R \sin(e_\psi) \simeq -k_\Omega \dot{\psi}_d, \quad |e_\psi| \simeq \arcsin(1/6) = 9.59^\circ$$

式（11）

代入kr=3、komega=1及目标角速度大小0.5 rad/s，估计误差幅值为9.59°，与实测9.524°接近。这解释了一个可能机制，不是对全部误差的严格辨识。

表20 结论的适用范围

现有证据	能够说明	不能推出
44.64秒稳定绕圆	该录制段内误差未持续发散	所有初始状态都稳定
高度与半径有小偏差	存在持续跟踪偏差	一定全由空气阻力造成
参数调整后晃动减轻	当前配置能完成任务	这组参数对所有机器最优
截图、视频、bag一致对应	过程和统计可以核对	单张截图等于完整飞行证明

可以怎样继续改

表21 后续改进建议，尚未实施

改进	为什么	怎样比较
目标角速度前馈	减少持续转向中的朝向滞后	保持轨迹和增益相同，比较yaw误差
阻力补偿	减少已建模外力造成的持续偏差	先核对模型，再做开关对比
重复试验与时延检查	确认结果可重复并解释短时波动	记录相同初值、消息时间及完整曲线

5 结论

本实验补全控制器的11处核心逻辑，并通过三阶段调参完成定点悬停和连续绕圆。最终参数为kx=2、kv=3、kr=3、komega=1。44.64秒录制段覆盖约3.55圈，三维位置RMSE为10.52厘米，偏航RMSE为9.524°。飞行基本稳定，但高度、半径和朝向仍有偏差。

最重要的经验是先检查坐标、旋翼编号和单位，再调参数。原始工程能编译，不代表控制器已经补完；看到一个漂亮的圆，也不代表没有误差。截图用于说明过程，视频用于检查连续运动，bag用于计算整段指标。

表22 实验交付材料

交付材料	作用
deliverable_1.txt	D1五问答案
deliverable_2.pdf	英文D2理论推导
controller_pkg/	完整可编译控制器包
RViz视频	展示连续绕圆
本报告及分析材料	说明实现过程、真实结果与局限

D2B只做了推导，未作为飞行改进实施。没有新增未记录的试验，也没有把理想估算当成实测结果。全部8张原始截图已在正文相关位置使用。

参考文献

[1] MIT VNAV. Lab 3 Exercises, 2023. https://vnav.mit.edu/labs_2023/lab3/exercises.html
[2] MIT-SPARK. VNAV-labs, VNAV2021 release. https://github.com/MIT-SPARK/VNAV-labs/releases/tag/VNAV2021
[3] Lee T, Leok M, McClamroch N H. Geometric Tracking Control of a Quadrotor UAV on SE(3). IEEE CDC, 2010: 5420-5425.
[4] ROS geometry2, noetic-devel. https://github.com/ros/geometry2/tree/noetic-devel

原图来源：hbyhby123/vnav-personal，lab3/data/raw/screenshots。逐样本数据、指标和分析脚本位于lab3/data/analysis。实验截图与原始记录的对应关系可直接核对。